SmartHealth

Intelligent Healthcare Operations Platform

Product Requirements Document — Part B

Gulsicka | August 2026

1. Product Overview

Part B extends SmartHealth with a GenAI layer delivered as a new ai-service microservice. It adds admin-facing conversational AI over the platform's own data, automated report and summary generation, AI-drafted patient/staff communications, and the analytics needed to measure how the AI layer is used — all built on top of the services delivered in Part A.

The design follows one governing principle throughout: the LLM is used for language, never for arithmetic. All totals, counts, and percentages are computed deterministically in Python at request time; the LLM only narrates from those verified numbers via a shared tool-calling agent, and pgvector is scoped to store only ingested document content — never entity or report data.

New Service

- ai-service — retrieval-augmented chat, report generation, AI-drafted communications, document ingestion, all backed by pgvector and a shared agentic tool-calling loop

2. Key Use Cases

UC-11 AI Chat Assistant

- Admin submits a natural-language question to /chat
- ai-service binds a scoped tool library to the LLM — single-entity lookups (patient, provider, clinic, department, user, appointment), appointment-by-patient/provider/clinic lookups, and a document-search tool — and the model decides which to call for a given question, instead of the query being parsed for an entity ID up front
- The answer streams over Server-Sent Events via a shared ReAct tool-calling loop (also used by /generate/communication and /generate/report); every round is streamed live, including visible "[calling get_patient...]" status events, so tool calls are visible as they happen
- Any ID surfaced by a tool result (department_id, user_id, clinic_id) is resolved through the matching tool before being shown to the user; the underlying Groq model is configurable via the GROQ_MODEL environment variable
- An ai.chat Kafka event is published on completion or failure, feeding AI usage analytics

UC-12 Report & Summary Generation

- Admin requests one of four report types for a given date: daily appointment summary, department utilization, patient engagement, or executive operational snapshot
- Each report type has a dedicated tool (e.g. get_daily_appointment_stats, get_department_utilization_stats) that fetches live data from provider-service, patient-service, appointment-service, and auth-service and computes the exact statistics in Python at request time — nothing is cached or precomputed ahead of time
- The report agent is required to call the relevant stats tool first and use its numbers exactly as returned; the LLM only narrates the report, it never calculates, estimates, or re-derives a total or percentage itself
- Because statistics are computed live per request, there is no ingestion step or staleness window for report data — numbers always reflect the current state of the underlying services

UC-13 AI-Generated Patient/Staff Communications

- Admin requests a follow-up message, service recommendation, preventive care suggestion, or operational assistance note for a specific patient, provider, or clinic
- The requested patient_id, provider_id, or clinic_id is passed to the same shared tool-calling agent as an instruction; the agent uses its entity lookup tools to fetch the real, current record and drafts the message from that live data — no pre-synced data involved

UC-14 Appointment Reminder Generation

- Given structured appointment, patient, provider, and clinic details and a reminder type (day-before or hour-before), the LLM drafts a short, friendly reminder with no placeholders
- Day-before and hour-before reminders are scheduled automatically via Temporal Schedules — a one-shot workflow is scheduled to fire at the exact reminder time when an appointment is created, and the schedule is cancelled if the appointment is later cancelled

UC-15 Document Ingestion & Retrieval

- Admin uploads a PDF via /ingest/pdf (or raw text via /ingest); pgvector stores only this ingested document content — no entity or report data is pre-synced into it
- PDF ingestion runs as a Temporal workflow (PdfIngestionWorkflow) with one activity per page, so parsing/embedding is retried per page and a single bad page doesn't fail the whole upload
- Each page is hashed and diffed against what's already stored: unchanged pages are skipped, changed pages have their old chunks replaced in one transaction, and pages removed from a shorter re-upload are cleaned up automatically
- Ingested content is retrieved by the chat/communication/report agent via a search_documents tool (cosine-similarity search), used for general hospital-knowledge questions rather than entity data

UC-16 AI Usage & Engagement Analytics

- analytics-service consumes ai.chat / ai.communication / ai.report events from the ai.events Kafka topic
- Redis counters and a persisted ai_interaction_events table are updated per event
- GET /analytics/ai (admin-only) reports AI Assistant Usage, Questions Asked/Answered, and Generated Communication Usage by type

3. Functional Requirements

AI Chat & Retrieval

- **FR-44** Admin-only /chat endpoint accepts a natural-language query and streams the answer via Server-Sent Events
- **FR-45** A scoped tool library (single-entity lookups only — patient, provider, clinic, department, user, appointment — never bulk/raw dumps) is bound to the LLM; the model decides which tool(s) to call for a given question
- **FR-46** A shared ReAct tool-calling loop is used by /chat, /generate/communication, and /generate/report, eliminating duplicated agent logic across endpoints
- **FR-47** Every round of the tool-calling loop is streamed live over Server-Sent Events, including visible status events for each tool call in progress, not just the final answer
- **FR-48** Any ID returned by a tool (department_id, user_id, clinic_id) must be resolved via the matching tool before being shown to the user; this is enforced explicitly in the system prompt
- **FR-49** The chat LLM is instructed to answer only from live tool results, never invented data, and to say so honestly when a lookup returns nothing

Report & Summary Generation

- **FR-50** /generate/report supports four report types: daily appointment summary, department utilization, patient engagement, executive operational snapshot
- **FR-51** All totals, counts, and percentages are computed deterministically in Python inside dedicated report-stats tools that fetch live data at request time; the LLM never performs arithmetic itself
- **FR-52** The report agent must call the appropriate stats tool before writing any numbers, and must use those numbers exactly as returned
- **FR-53** Per-provider, per-department breakdowns are precomputed for daily reports so the LLM is not required to group or tally raw appointment lists

- **FR-54** The system prompt explicitly forbids the LLM from inventing patients, providers, or departments not present in the supplied data
- **FR-55** Report statistics are always computed live per request — there is no ingestion step or staleness window for report data

AI-Generated Communications

- **FR-56** /generate/communication supports four types: follow-up, service recommendation, preventive care, operational assistance
- **FR-57** Requests are scoped to a specific patient, provider, or clinic via a typed ID in the request body; that ID is passed to the shared tool-calling agent, which fetches the live record itself rather than a pre-synced chunk
- **FR-58** /generate/reminder drafts a short reminder message (day-before or hour-before) from structured appointment, patient, provider, and clinic details, without placeholders
- **FR-59** Chat, report, and communication calls always record success/failure status as a Kafka event, even when the underlying LLM call fails

Document Ingestion & Retrieval

- **FR-60** PDF ingestion runs as a Temporal workflow (PdfIngestionWorkflow) with one activity per page; a single page's parsing/embedding failure does not fail the whole upload
- **FR-61** Pages are hashed and diffed per re-upload: unchanged pages are skipped, changed pages have their chunks replaced in a single transaction, and pages removed from a shorter re-upload are cleaned up (orphan-page deletion)
- **FR-62** Text chunks are embedded with the all-MiniLM-L6-v2 sentence-transformer model before storage
- **FR-63** /ingest (plain text) and /ingest/pdf (via the Temporal ingestion workflow) allow ad hoc ingestion of arbitrary text and PDF documents into the same pgvector store — the only data pgvector now holds

AI Analytics & Observability

- **FR-64** Every /chat, /generate/communication, and /generate/report call publishes an ai.events Kafka event carrying event type, status, and user ID
- **FR-65** GET /analytics/ai (admin-only) returns AI Assistant Usage, Questions Asked/Answered, and Generated Communication Usage broken down by type
- **FR-66** LangChain LLM calls are auto-instrumented via OpenTelemetry and exported to Jaeger for tracing of model latency, token usage, and errors
- **FR-67** Day-before and hour-before appointment reminders are scheduled via Temporal Schedules — a one-shot workflow per reminder is scheduled when an appointment is created and cancelled if the appointment is cancelled
- **FR-68** The tool library exposed to any agent is restricted to single-entity lookups and pre-verified computed-stat tools; no endpoint exposes a bulk/raw entity dump to the LLM

4. Non-Functional Requirements

ID	Category	Requirement
NFR-12	Scalability	ai-service embedding/generation workloads run independently of core booking services and scale without affecting appointment/patient/provider throughput
NFR-13	Accuracy	Numeric values in generated reports must match Python-computed ground truth exactly; LLM output is never treated as the source of truth for counts
NFR-14	Data freshness	Entity data (patients, providers, appointments) is fetched live per request, so there is no staleness window; only ingested PDF/document content depends on when it was last uploaded

ID	Category	Requirement
NFR-15	Auditability	Every AI interaction (chat, report, communication) is durably logged as a Kafka event and a database row, regardless of success or failure
NFR-16	Observability	LLM call latency, token counts, and errors are traceable end-to-end via OpenTelemetry/Jaeger without manual span instrumentation
NFR-17	Access control	All GenAI endpoints (chat, report, communication, reminder, AI analytics) are restricted to the admin role
NFR-18	Configurability	The Groq model identifier is externalised to an environment variable (GROQ_MODEL) so the underlying LLM can be swapped without code changes

5. Delivery Timeline & Milestones

Milestone	Week	Part	% Alloc.	Expected Start	Expected End	Actual Completion
Part B	Week 4	Part B	1	27-Jul-2026	03-Aug-2026	29-Jul-2026
Part B	Week 5	Part B	1	03-Aug-2026	10-Aug-2026	06-Aug-2026
Part B	Week 6	Part B	1	10-Aug-2026	19-Aug-2026	19-Aug-2026

Week 4 — 27-Jul-2026 to 03-Aug-2026 (completed 29-Jul-2026)

- Built the RAG chat pipeline: pgvector store, all-MiniLM-L6-v2 embeddings, /chat with SSE streaming
- Added entity-aware retrieval: source_prefix extraction from queries, boundary-safe chunk filtering
- Built /generate/report: deterministic Python statistics stored as pure-data chunks, prompt framing applied only at request time
- Fixed LLM hallucination issues (miscounted totals, invented providers) by removing distractor data and precomputing breakdowns

Week 5 — 03-Aug-2026 to 10-Aug-2026 (completed 06-Aug-2026)

- Wired /generate/communication and /generate/reminder to real chunk data with per-entity scoping
- Extended /ingest/sync to precompute and store report statistic chunks alongside entity chunks
- Added OpenTelemetry auto-instrumentation for LangChain LLM calls, exported to Jaeger
- Added ai.events Kafka topic, analytics-service consumer, and GET /analytics/ai (AI Assistant Usage, Questions Asked/Answered, Generated Communication Usage)
- Externalised the Groq model identifier to the GROQ_MODEL environment variable

Week 6 — 10-Aug-2026 to 19-Aug-2026 (completed 19-Aug-2026)

- Migrated /chat, /generate/communication, and /generate/report to a shared, agentic tool-calling loop with a scoped tool library — no more query parsing, chunk pre-fetching, or /ingest/sync
- Rebuilt PDF ingestion as a Temporal workflow (PdfIngestionWorkflow) with per-page hashing, diffing, and orphan-page cleanup, replacing whole-file re-embedding
- Replaced sleeping reminder workflows with Temporal Schedules — one-shot day-before/hour-before workflows scheduled per appointment and cancelled on cancellation
- Scoped pgvector to store only ingested PDF/document content; report statistics are now computed live per request instead of cached

6. Traceability Matrix

FR ID	Requirement Summary	Use Case(s)	Service(s)	Week
FR-44	SSE-streamed chat endpoint	UC-11	ai-service	Week 4

FR ID	Requirement Summary	Use Case(s)	Service(s)	Week
FR-45	Scoped tool library bound to chat LLM	UC-11	ai-service	Week 6
FR-46	Shared ReAct tool-calling loop (chat/communication/report)	UC-11	ai-service	Week 6
FR-47	Live SSE streaming of every tool-loop round	UC-11	ai-service	Week 6
FR-48	Tool-surfaced ID resolution before display	UC-11	ai-service	Week 6
FR-49	Tool-result-grounded chat answers	UC-11	ai-service	Week 6
FR-50	Four report types	UC-12	ai-service	Week 4
FR-51	Live-computed statistics via report-stats tools	UC-12	ai-service	Week 6
FR-52	Report agent must call stats tool before writing numbers	UC-12	ai-service	Week 6
FR-53	Precomputed provider/department breakdowns	UC-12	ai-service	Week 4
FR-54	Anti-fabrication system prompt rule	UC-12	ai-service	Week 4
FR-55	Report stats always computed live (no staleness)	UC-12	ai-service	Week 6
FR-56	Four communication types	UC-13	ai-service	Week 5
FR-57	Typed ID passed to tool-calling agent for live lookup	UC-13	ai-service	Week 6
FR-58	Placeholder-free reminder drafting	UC-14	ai-service	Week 5
FR-59	Always-recorded call status events	UC-12, UC-13	ai-service, analytics-service	Week 5
FR-60	Temporal workflow PDF ingestion (per-page activities)	UC-15	ai-service, temporal-workflow	Week 6
FR-61	Per-page hash diff + orphan-page cleanup	UC-15	ai-service	Week 6
FR-62	all-MiniLM-L6-v2 embeddings	UC-15	ai-service	Week 4
FR-63	Ad hoc text/PDF ingestion	UC-15	ai-service	Week 4
FR-64	ai.events Kafka publishing	UC-16	ai-service, analytics-service	Week 5
FR-65	GET /analytics/ai endpoint	UC-16	analytics-service	Week 5
FR-66	OpenTelemetry/Jaeger LLM tracing	UC-11, UC-12, UC-13	ai-service	Week 5
FR-67	Temporal Schedule reminder triggering	UC-14	ai-service, temporal-workflow	Week 6
FR-68	Scoped-only tool library (no bulk dumps)	UC-11, UC-12, UC-13	ai-service	Week 6